Internship Report DeVi Comfort. 2026; 1(1): 1–15

DEVELOPING A LIVE 3D TELEMETRY VISUALIZATION FOR STAIRLIFT FAULT DIAGNOSIS

Nout Mulder
DeVi Comfort • Engineering • Opmeer

Abstract. This research addresses the challenge of transforming raw stairlift telemetry into a stable, accurate live 3D visualization for fault diagnosis. The core technical problem is the mapping of digital pulse-encoder readings, polled over a high-latency Bluetooth link, onto a curved 3D rail geometry with segment-dependent encoder-to-distance ratios, with sub-rendered-frame smoothness on a 60 fps display. A "virtual ghost" prototype was developed iteratively and validated on physical hardware with two calibration runs (bottom-to-top and top-to-bottom) plus a stationary baseline. The pipeline- integration error across 10 measurements (five reference markers, each read in both drive directions) stayed within 0.7 mm (mean 0.39 mm, SD 0.22 mm, well within the 5 mm tolerance for the position-accuracy criterion), the chair on screen tracked the physical chair without perceptible lag across both runs despite individual Bluetooth round-trips that occasionally exceeded the 100 ms threshold (mean 89 ms over 5295 polls), and the displayed position interpolated smoothly through the sensor stream without perceptible step-jumps. The results show that a processing pipeline combining non-linear encoder mapping, recursive state estimation with dead-reckoning, and smooth tangent-based orientation reconstruction produces a stable, accurate live 3D representation of stairlift position and fault state.

1 Introduction

DeVi Comfort designs and manufactures stairlift systems and develops the accompanying control software in-house. A stairlift consists of a chair unit that travels along a custom-bent rail mounted to a staircase. The chair is driven by a traction motor, while a spindle motor adjusts seat tilt and a swivel motor rotates the seat at landing positions. During operation, embedded controllers continuously produce telemetry: traction encoder pulses, spindle encoder values, swivel angles, and system status bits including error codes. For service, maintenance, and further development, engineers need to quickly understand the current state of a stairlift: where the chair is positioned on the rail, whether it is moving correctly, and what deviations or faults have occurred.

Currently, fault information from the controller is presented as textual error codes and numeric status values. While technically correct, these require significant domain knowledge to interpret spatially. An error code indicating a position deviation does not convey where on the rail the deviation occurred, in which direction, or how large the discrepancy is.

The underlying technical challenge is twofold. The raw telemetry data (encoder pulse counts, spindle values, swivel angles) must be mapped onto a three-dimensional rail geometry. This rail contains straight sections, vertical bends, and horizontal bends, each consuming a different number of encoder pulses per millimeter of travel. Additionally, the sensor readings arrive at irregular intervals over a wireless serial link with noise, latency, and occasional packet loss. The visualization must bridge the gap between these asynchronous measurements and a continuous, smooth spatial representation that updates at display frame rate.

A live 3D visualization of stairlift position and movement would have direct practical value. Service engineers could visually identify whether the chair has lost its expected position, is stuck in a bend, or exhibits abnormal tilt. This would reduce diagnosis time and minimize unnecessary on-site visits. For the development team, a live telemetry view would enable rapid verification of firmware behavior during testing and commissioning of new rail configurations.

The goal of this research is to determine which data processing and mathematical models are needed to transform live stairlift telemetry into a stable and accurate 3D visualization. Whether such a visualization ultimately accelerates fault diagnosis compared to text-based error codes is a practical motivation for the work, but lies outside the scope of this report; the central research question is therefore restricted to what can be deductively established:

What data processing pipeline and mathematical models are required to transform live stairlift telemetry into a stable, accurate 3D visualization?

The research follows an iterative prototyping approach based on the spiral model [1]. Each iteration consists of analysis, design, implementation, and validation against practical scenarios on physical hardware. The primary methods are formal mathematical analysis, architectural design, and experiment with measurable accuracy criteria.

Section 2 presents the problem statement, structured as a problem analysis, prior work survey, and the resulting research questions. Section 3 describes the research methods per subquestion. Section 4 presents the findings. Section 5 concludes with answers to the research questions and recommendations.

2 Problem Statement

2.1 Problem Analysis

2.1.1 Sensor-to-Spatial Mapping in Mechatronic Systems

The challenge of transforming discrete sensor readings into a continuous spatial representation is well-established in mechatronic system diagnostics [2]. In the broader context of digital twin research, Bofill et al. identify real-time sensor data integration as one of the primary technical barriers to effective fault monitoring [3]. The fundamental difficulty is that sensors produce scalar values (counts, voltages, status bits) at discrete intervals, while a spatial visualization requires a continuous 3D position and orientation that updates at display frame rate.

In robotics and vehicle tracking, this problem is typically simplified by the assumption that sensor readings map linearly to spatial coordinates [4]. A rotary encoder on a linear actuator, for example, produces pulses proportional to displacement, and the mapping from pulse count to position is a single constant. This assumption breaks down when the trajectory is non-linear: on a curved path, the relationship between encoder pulses and spatial displacement depends on the local curvature [5].

2.1.2 Non-Linear Encoder Mapping on Curved Rails

A stairlift rail is composed of straight sections, vertical bends (pitch changes), and horizontal bends (yaw changes). The mechanical drive system uses a traction motor with an incremental encoder to measure displacement [6]. Due to the geometry of the rail bending mechanism, each segment type consumes a different number of encoder pulses per unit of physical travel. A vertical upward bend, for instance, requires fewer pulses per millimeter of arc length than a downward bend, because the traction wheel follows the inner or outer radius of the curve respectively.

This means that a single scalar encoder reading cannot be converted to a 3D position without a model of the full rail topology. The mapping must account for the cumulative pulse cost of each preceding segment, making it a piecewise non-linear function rather than a simple linear scaling. Without such a model, position errors accumulate at every bend transition, growing proportionally to the number of bends in the rail.

2.1.3 State Estimation from Asynchronous Telemetry

The telemetry link between the stairlift controller and the visualization application introduces additional challenges. The Bluetooth Classic Serial Port Profile [7] provides a virtual serial connection at approximately 20 Hz; the round-trip time at the application layer depends on link-layer latency plus controller response time and was measured in the validation runs of Section 4.4. The visualization, however, must render at 60 fps (16.7 ms per frame).

This creates a temporal mismatch: new sensor data arrives many render frames apart, and frames between measurements must be filled with predicted values. If the most recent measurement is simply held constant until the next arrives, the display exhibits visible staircase-like jumps at the sensor sample rate. The traction encoder used by the stairlift is a digital incremental pulse-counter [5]: it does not produce per-sample quantization noise on a stationary chair, but it does produce sampled positions at a rate well below the display refresh, so the gap-filling problem (not noise rejection) is the dominant artifact source for this visualization.

This frames the technical requirement precisely: the visualization needs a continuous position-and-velocity estimate that can be advanced at display frame rate from samples arriving several times less often, and that can bridge the interval when a sample is late or missing. The required property is therefore gap-filling and velocity estimation between samples, not noise rejection on the individual sample. Which class of estimator satisfies this requirement, and at what cost, is a solution question addressed in the prior work survey (Section 2.2).

2.1.4 Multi-Axis Orientation from Independent Sensors

The stairlift chair has four independent rotational degrees of freedom: yaw (direction along the rail), pitch (rail inclination), seat tilt (spindle motor), and seat rotation (swivel motor). Each is measured by a separate sensor with its own update rate and noise characteristics. Reconstructing the full 3D orientation of the chair requires combining these independent streams into a single consistent rotation that stays well-defined for every reachable chair configuration, including those in which two of the rotation axes momentarily align. Whether a particular orientation representation can guarantee that property is a solution question, addressed in Section 2.2.

2.1.5 Summary of Technical Subproblems

Three interrelated technical subproblems emerge from this analysis:

1. **Non-linear encoder-to-position mapping:** Converting a scalar encoder count into a normalized position along a 3D curve with segment-dependent pulse costs.
2. **State estimation:** Producing a smooth, low-latency estimate of position and velocity from noisy, asynchronous measurements arriving at a lower rate than the display refresh.
3. **Multi-axis orientation reconstruction:** Combining four independent rotational sensor streams into a singularity-free 3D orientation.

2.2 Prior Work

2.2.1 Encoder-to-Position Mapping on Non-Uniform Paths

Converting an incremental encoder count to a position along the travel path is, for a uniform path, a solved problem: the encoder produces pulses proportional to displacement and a single scale constant suffices [6, 4]. Three

solution directions exist for the non-uniform case identified in Section 2.1.

A **single linear scale factor** multiplies the pulse count by one constant mm-per-pulse value, the standard linear-actuator approach [4]. It is $O(1)$ per frame and needs no model of the path, but is correct only while the pulses-per-millimetre ratio is constant; [5] shows this assumption fails where the local path geometry varies, so on a bent rail the linear factor accumulates a position error at every bend transition. It is therefore rejected as the primary mapping.

A **closed-form arc-length parameterization** integrates the encoder-to-arc-length relation analytically when the path is a single analytic curve [8]. This is exact and compact, but a closed-form inverse exists only for a single uniform segment type (constant curvature). The stairlift rail mixes straight, vertical-bend and horizontal-bend segments with different per-segment pulse costs, so no single closed-form inverse covers the whole rail; the approach is usable only per segment, not rail-wide.

A **precomputed piecewise calibration map** tabulates, at each segment boundary, the cumulative encoder count against the cumulative Euclidean distance and interpolates within a segment, reducing every per-frame query to a binary search over a static table [6]. It handles arbitrary per-segment pulse costs, amortises the path integration to once per rail, and degrades gracefully (piecewise-linear) between boundaries. Its cost is that the full rail topology must be known up front — which it is, since the controller stores the rail as typed blocks. This direction is selected and derived in Section 4.2.

2.2.2 State Estimation

For state estimation from noisy sensor data, three approaches are commonly used in embedded and robotics applications: moving average filters, complementary filters, and Kalman filters.

A **moving average filter** computes the mean of the last N samples and outputs it as the filtered value. Smith shows that for purely time-domain noise reduction on stationary or slowly varying signals, moving averages are close to optimal and have the advantage of computational simplicity [9]. They are therefore a strong choice for applications where the input signal changes slowly relative to the sampling rate and no prediction is required between samples. However, moving averages introduce a latency proportional to the window size (effectively delaying the signal by $N/2$ samples), cannot extrapolate between measurements, and perform poorly on transient or fast-changing signals. For stairlift telemetry, where per-channel sensor readings arrive several times less often than the 60 fps display refresh, pure moving-average smoothing cannot fill the inter-sample gap.

A **complementary filter** combines two sensor sources with complementary frequency characteristics, typically using a low-pass filter on one source and a high-pass filter on the other [10]. This is the standard approach in IMU fusion, where slow, drift-free accelerometer data is combined with fast, noisy gyroscope data to estimate attitude. Complementary filters are computationally light and do not require a statistical model of the noise. They are therefore preferred in cost-constrained or high-rate embedded systems with multiple complementary sensors. However, they assume that two sensor sources are available with complementary frequency content, which is not the case for the stairlift: the traction encoder is the only direct position sensor, and its noise is not separable by frequency from its signal.

The **Kalman filter** maintains a probabilistic estimate of system state (position and velocity) and updates it with each new measurement, optimally balancing prediction uncertainty against measurement noise [11, 12, 6]. It combines state estimation and prediction in a single framework, which makes it a natural fit when measurements arrive at a lower rate than the required output. The cost is higher computational complexity and the need to tune process and measurement noise parameters. For the stairlift application, the Kalman filter is selected because it is the only approach of the three that can both estimate a continuous velocity from the discrete pulse-counter stream and predict position between measurements, both of which are required to feed a 60 fps display from a per-channel sensor stream that arrives several times more slowly than the display refresh, without visible staircase-like jumps at the sample rate.

2.2.3 Spline Interpolation for Smooth Trajectories

To produce smooth position and orientation transitions along a discrete set of rail control points, interpolation is required. Three families of splines are candidates.

Linear interpolation between adjacent points produces sharp direction changes at segment boundaries (no C^1 continuity). It is computationally cheap and is the right choice when the interpolated quantity is expected to change piecewise linearly, or when downstream processing does not need a continuous tangent. For visual orientation of a chair along a continuously curved rail, the discontinuous tangent of linear interpolation would cause the chair to visibly "snap" at each rail point.

B-splines (and their rational generalization, NURBS) are the de facto standard in CAD and industrial curve design. They provide arbitrary continuity (C^k) through the choice of degree, and local control (moving one control point affects only a bounded region of the curve) [13]. They are preferred when the curve itself is being designed (CAD surfaces, font glyphs, animation paths) and the control points are construction handles rather than fixed waypoints. Their disadvantage for this application is that the curve does *not* pass through the control points, so it does not correspond to the physical rail geometry: the ghost chair would follow a smoothed approximation of the rail rather than the rail itself.

Catmull-Rom splines interpolate through each control point and provide C^1 continuity at those points [14]. They are the natural choice when the control points are fixed physical waypoints (as is the case for the rail) and the primary goal is a smooth tangent for orientation purposes. They are selected for this work specifically as the *tangent* source for chair orientation; the chair *position* along each segment is computed by linear interpolation between adjacent rail control points, which preserves the on-rail constraint trivially because both linear

and Catmull-Rom curves pass through the control points. The combined scheme (linear position, Catmull-Rom tangent) gives smooth chair orientation without introducing the off-rail excursions that a full Catmull-Rom positional curve would produce between sparsely sampled points.

2.2.4 Rotation Representation

Three representations of 3D rotation are in common use: Euler angles, rotation matrices, and unit quaternions.

Euler angles (roll, pitch, yaw) are human-readable and require only three parameters. Diebel notes that for systems with limited rotation ranges, or where only a single rotation axis is actively used, Euler angles are a reasonable and computationally inexpensive choice [15]. Many industrial applications (e.g., single-axis servo control, simple tilt sensors) use Euler angles without issue. The well-known drawback is gimbal lock: when two rotation axes align, one degree of freedom is lost and the inverse mapping becomes singular. For the stairlift, with four independent rotational axes that can interact (rail yaw, rail pitch, seat tilt, seat rotation), the risk of gimbal lock during certain rail configurations is real.

Rotation matrices (3×3 orthogonal matrices) are free of singularities and compose cleanly, but require nine parameters with six constraints, are costly to interpolate smoothly, and accumulate numerical drift (loss of orthogonality) over many composed rotations. They are standard in computer graphics shaders, where the GPU performs matrix operations natively.

Unit quaternions (four parameters, one constraint) are free of gimbal lock, compose compactly, and support smooth spherical linear interpolation [16]. They are the standard in real-time graphics and robotics for full 3D orientation [17, 15]. For the stairlift chair, where four independent rotations must be composed every frame without visual artifacts, quaternions are selected to avoid the gimbal lock risk of Euler angles and the numerical drift of rotation matrices.

2.2.5 Real-Time 3D Engine Selection

The renderer layer of the pipeline (Section 4.1) imposes four *mandatory* deployment constraints on the 3D engine, each a hard requirement rather than a weighted preference: (C1) configurability for custom shaders, (C2) cross-platform export to desktop and Android 12+ from a single source tree, (C3) a licence compatible with closed-source commercial deployment, and (C4) native glTF binary import to match DeVi Comfort's CAD pipeline [18]. Because these are constraints to be satisfied rather than trade-offs to be balanced, the selection is a feasibility screen (does an engine meet every constraint?) rather than a weighted multi-criteria trade-off; a partial match on a mandatory constraint counts as a failure. Table 1 screens four candidates against them.

Table 1: Feasibility screen of real-time 3D engines against the four mandatory deployment constraints defined in Section 2.2.5. "+" indicates the constraint is fully met, "=" partially, "−" not met; only an engine meeting all four is feasible.

Engine	C1	C2	C3	C4
Godot 4.6 [19]	+	+	+	+
Unity 6 [20]	=	+	=	+
Unreal Engine 5 [21]	+	=	=	+
Three.js + WebGL [22]	=	−	+	+

Godot 4.6 is selected: it is the only candidate satisfying all four mandatory constraints. Its MIT licence [19] avoids royalty and seat-fee risk; its export pipeline compiles a single source tree to Windows, Linux, and Android. Unreal was excluded on licence (5 % royalty above a per-product annual revenue threshold [21]) and mobile export friction; Unity on its seat-fee and revenue-threshold model [20] combined with licence-stability risk after the 2023 runtime-fee announcement [23]; Three.js on lack of a native Android build target, which would force a WebView wrapper and break the gyroscope/accelerometer access required for the compass tracker (Section 4.4).

2.2.6 Existing Work at DeVi Comfort

No spatial visualization of stairlift telemetry existed at DeVi Comfort prior to this research; fault output is described in Section 1. The IPC (Inter-Process Communication) infrastructure connecting the configurator application to the lift controller was limited to rail upload and download, so the input layer of the proposed pipeline (live telemetry polling) had no existing implementation to build on.

2.3 Research Questions

The problem analysis identified three technical subproblems (non-linear encoder mapping, state estimation, and multi-axis orientation reconstruction), while the prior work survey identified candidate approaches for each. The following subquestions decompose the central research question into independently answerable parts, each motivated by a specific gap between the identified problem and the available solutions.

1. *What data pipeline architecture transports live telemetry from the controller to a 3D rendering environment within real-time constraints?* This question subsumes the identification of which firmware messages (out of over 140 defined types) carry spatially interpretable information, because the input layer of the pipeline is precisely that subset. The Bluetooth serial link introduces latency, noise, and packet loss, which the pipeline must handle while keeping pace with the sensors as closely as the shared link allows.
2. *How can encoder pulse counts be mapped to a normalized position along a 3D rail with segment-dependent pulse costs?* As established in Section 2.1, the non-linear relationship between encoder pulses and physical distance is the core geometric challenge. No existing mapping exists for this rail type.

3. *Which state estimation and orientation models produce a stable live 3D representation from noisy telemetry?* The prior work survey identified several candidate models per subproblem. This question addresses the formal derivation, parameterization, and empirical evaluation of the selected models for this specific application.

The empirical evaluation of each subquestion against measurable thresholds is treated as a validation methodology that applies across all three, rather than as a separate research question. It is described in Section 3.4.

3 Methods

Each subquestion is addressed with one or more research methods. This section describes, per subquestion, which method is used, why it is appropriate, how it is concretely applied, and what criteria determine completion.

3.1 Data Pipeline Architecture (Design + Documentation Analysis)

Method: Architectural design, combined with formal analysis of the firmware protocol specification to define the input layer.
Why this method: The telemetry pipeline spans multiple layers (wireless transport, binary protocol parsing, polling, state estimation, rendering) and must poll the position-relevant channels as fast as the shared Bluetooth link allows while keeping the round-trip time within a budget compatible with a 60 fps render loop. A structured architectural design ensures that each layer's contribution to latency and reliability is identifiable. The input to that pipeline is a subset of the firmware's > 140 message types, which is determined by documentation analysis rather than observation of live traffic: the input is the complete firmware enumeration, not sampled message frames.
Approach: The architecture is specified as a layered pipeline (transport, protocol parser, telemetry poller, state estimator, 3D renderer). Each layer is documented with its inputs, outputs, and failure modes. As input to the design, the message enumeration in the controller firmware source is extracted and each message type is classified along three axes: (1) data content, (2) update rate, and (3) spatial interpretability.
Why this approach: A five-layer decomposition is chosen (rather than a flat producer/consumer or a two-tier client/renderer) because each identified failure mode (bit-level corruption, message-level corruption, connection drops, sensor noise, frame drops) maps cleanly onto exactly one layer. A flatter split would force the renderer to handle protocol errors or the transport to know about state estimation, collapsing concerns that the timing budget already separates. The three-axis message classification is chosen over a single-axis "relevant yes/no" because content selects parsing logic, rate selects polling strategy, and spatial interpretability selects relevance to the visualization.
Instrument (when done): The design is considered sufficient when: (a) every message ID in the firmware enumeration has been classified, with the subset relevant to the visualization pipeline explicitly identified with rationale and documented at byte-level for any message with a position, angle, or status bit field; and (b) the chair on screen visually tracks the physical chair during steady-state motion without perceptible lag, as observed during the validation runs of Section 3.4. This is an observational criterion: the inter-sample Bluetooth round-trip is reported as a context metric, but the relevant end-user property is the visible lag after the Kalman filter and dead-reckoning have compensated for inter-sample gaps. A naïve single-sample-latency threshold would not capture this, because the pipeline architecture explicitly masks per-sample latency at the display side (see Section 4.3).

3.2 Encoder-to-Position Mapping (Formal Analysis)

Method: Formal mathematical analysis.
Why this method: The encoder-to-position mapping is a deterministic geometric problem: given a rail composed of segments with known curvature and known pulse costs per segment type, the mapping can be derived analytically. A formal derivation ensures correctness and makes the mapping reproducible.
Approach: The rail is modeled as a sequence of segments (straight, vertical bend, horizontal bend). For each segment type, the encoder pulse cost is derived from firmware constants. Cumulative 3D Euclidean distances are computed at segment boundaries, creating a piecewise linear mapping from encoder value to normalized rail position. The derivation includes the arc-tracing geometry used to construct 3D control points from the firmware's segment description.
Why this approach: Storing a precomputed piecewise mapping at segment boundaries (instead of integrating the curve at every frame) is chosen because the mapping is static once the rail is known, while the encoder reading changes every frame; amortising the integration once per rail keeps the per-frame cost to a single binary search. The alternative of a single closed-form encoder-to-arc-length function would only exist for a single uniform segment type, which does not hold here.
Instrument (when done): The formal derivation is considered complete when: (a) the mapping is defined for all segment types in named equations traceable to firmware constants, and (b) the resulting displayed position deviates by less than 5 mm from the physically measured chair position at every reference point (station boundaries, each bend entry, each bend exit) on rails up to 5 m, and less than 10 mm on longer rails, across the validation runs of Section 3.4.

3.3 Mathematical Models (Formal Analysis)

Method: Formal mathematical analysis.
Why this method: The state estimation, interpolation, and rotation models are grounded in established mathematical theory (Kalman filtering, Catmull-Rom splines, quaternion algebra). Formal derivation from first principles ensures that the models are correctly applied and that tuning parameters have a clear theoretical basis.
Approach: Three models are derived:

- A 2D constant-velocity Kalman filter for each telemetry axis (traction, spindle, swivel), including the process noise matrix derivation, measurement update equa-

tions, and inter-measurement dead-reckoning extrapolation.
- Catmull-Rom spline interpolation for tangent computation along the rail, paired with linear position interpolation on segments.
- Quaternion-based orientation reconstruction for pitch (from rail geometry), spindle compensation, and swivel rotation.

Why this approach: The three models are derived independently, each with its own state and its own update equations, rather than as a single fused state vector. This is chosen because the three axes have distinct noise characteristics (traction encoder quantization, spindle lookup interpolation, swivel digitisation) and distinct time bases; a single fused filter would require a shared process noise model that averages away these differences. Per-axis constant-velocity filters allow tuning (σ_a, σ_m per axis, Table 4) to reflect each sensor's actual behaviour.
Instrument (when done): Each model is considered correct when (a) every step of the derivation is presented in named equations traceable to the cited sources, and (b) the display position interpolates visually smoothly between sensor samples; no perceptible step-jumps are observed at the per-channel application-layer sample rate ($\sim$4–5 Hz, bounded by the shared-link round-trip; see Section 4.1) on the 60 fps render loop, across the validation runs of Section 3.4. The Kalman filter in this pipeline functions primarily as a *temporal* interpolator and velocity estimator for the render loop, not as a noise-rejection filter: the traction encoder is a digital pulse-counter that produces no value-jitter on the sample itself, so the relevant quality is the smoothness of the filtered output as a continuous-time signal that fills the $\sim$200 ms gaps between sensor updates. Singularity-freeness of the quaternion composition is a theoretical property of the formalism [16, 17] and is therefore not separately measured; the absence of visible chair snapping during the validation runs is a sufficient observational check.

3.4 Validation Approach (Experiment)

The instrument criteria of Sections 3.1–3.3 are bundled under the labels DV1, DV2, and DV3 (*deelvalidatie* 1–3), one per research subquestion. Each is split into an analytical sub-criterion (a) (derivation traceable to literature or firmware constants) and an empirical sub-criterion (b) (measured on a physical calibration run). The empirical sub-criteria are listed in Table 2; the analytical sub-criteria are contained in the originating method sections. All empirical sub-criteria are evaluated through the same physical experiment described in this section, rather than through separate experiments, so all three subquestions can be evaluated from one set of calibration runs.

Table 2: Empirical validation criteria DV1(b)–DV3(b) with their originating method section and pre-registered pass threshold. Sub-criterion (a) (analytical traceability) is stated in the originating method section.

ID	Origin	Threshold (pre-registered)
DV1(b)	Section 3.1	BT round-trip < 100 ms (reframed observational, see Limitations)
DV2(b)	Section 3.2	Display vs. physical position < 5 mm on rails ≤ 5 m; < 10 mm on longer rails
DV3(b)	Section 3.3	Kalman output jitter $\geq 10\times$ smaller than raw (reframed observational, see Limitations)

Method: Experiment on physical stairlift installations with predefined pass/fail thresholds.
Why this method: The mathematical models and pipeline architecture must be verified under real-world conditions where mechanical tolerances, Bluetooth variability, encoder quantization, and sensor noise all interact. Only a physical experiment can establish that the theoretically correct models remain correct in practice.
Approach: Calibration runs are executed on multiple stairlift installations with different rail configurations (varying length, number of bends, and bend types). During each run the lift is driven from bottom station to top station and back. The prototype logs raw encoder values, filtered positions, angle corrections at each bend, render frame rate, and Bluetooth round-trip time. Position error is measured by comparing the displayed position at known reference points (station boundaries, bend start/end) against the physical position of the chair using the measurement procedure described in Section 4.4.
Why this approach: Thresholds are defined before the runs, not after, and each is tied to one of the DV1–DV3 instrument criteria. Measuring at fixed reference points (station boundaries and bend entries/exits) rather than averaging over the full run is chosen because these are the points at which a mapping fault would be visible to an operator; an average over the full rail would dilute a localised error.
Instrument (when done): A run is reported with measurements against every DV1–DV3 criterion that requires empirical evidence. Pass/fail is assigned per criterion as defined in the originating method section. Runs that fail on any criterion are reported with the cause.

Observational notes recorded alongside the measurements:

- O1 (pipeline robustness): the pipeline sustains a continuous drive over the full rail in both directions without crashes, freezes, or UI corruption.
- O2 (visual quality through bends): the chair on screen tracks the rail through bends without visible snapping or jumps at segment boundaries.
- O3 (visible motion smoothness during steady-state driving): while the chair drives between stations at typical pilot speed, the rendered chair on screen tracks the physical chair without perceptible step-jumps or lag.

This observation directly evaluates the end-user property that DV1(b) and DV3(b) aim to capture, in a way that the raw single-sample BT round-trip cannot.

4 Results

4.1 Data Pipeline Architecture

The input layer of the pipeline is the subset of the firmware's message enumeration that carries spatially interpretable information. Analysis of the stairlift firmware protocol reveals 145 defined message types (with IDs ranging from 0 to 221), of which six are directly relevant for live spatial visualization. These are classified in Table 3.

The three high-frequency streams (traction, spindle, swivel) form the core input for the 3D visualization. They are polled in an alternating pattern to share the single serial Bluetooth link: the fast loop is scheduled to tick at ∼20 Hz and alternates between traction and spindle requests on successive ticks, while the swivel coroutine polls on the same link at 50 ms intervals. The achievable rate is bounded by the application-layer round-trip time, which was measured at mean 89 ms over the validation runs (Section 4.4): because each request and its response must complete before the next request is issued on the shared link, the *aggregate* round-trip throughput is ∼10 Hz (5295 round-trips over the 524 s of driving), not the nominal 20 Hz. That aggregate is divided over the traction, spindle, swivel and status channels, so the per-channel traction arrival rate is correspondingly lower — on the order of ∼4–5 Hz, an inter-sample gap of ∼200 ms. The renderer reconciles this inter-sample gap with Kalman-based interpolation and dead-reckoning (Section 4.3); the larger-than-nominal gap is precisely what makes the inter-sample prediction load-bearing rather than cosmetic. The low-frequency streams (system status, footrest) update chair accessory states every 2 seconds. Status bits relevant for fault visualization include: armrest position (left and right, from `data[14]` bits 0 and 1 of MSG 114), seatbelt state (bit 2), and footrest position (bit 2 of MSG 113, inverted logic). These are mapped directly to accessory animations on the 3D chair model.

These six messages are transported from the lift controller to the renderer through a five-stage pipeline. Each stage has a single responsibility: receive data from the previous stage, process it, and pass the result to the next. This separation makes it possible to develop and test each stage independently. Figure 1 shows the five layers.

Transport layer. Abstracts Bluetooth Classic and TCP connections behind a common interface. The Bluetooth transport communicates with an HC-06 module (a low-cost serial-to-Bluetooth adapter) on the lift controller. Before each session, 40 null bytes are sent to flush the module's receive buffer. Message counters increment from 1 to 200 and wrap around.

Protocol parser. A finite state machine (FSM, a parser that moves through a fixed sequence of states for each incoming byte) [24] processes the incoming byte stream: `WAIT_START` → `MSG_NUMBER` → `COMMAND_ID` → `LENGTH` → `DATA` → `CHECKSUM`. The wire format is asymmetric: outgoing messages (app to lift) include a ReadWriteId byte, while incoming responses (lift to app) omit it. The byte layout is:

App → Lift: `[0x0A][Nr][Rw][Cmd][Len][Data...][Chk]`
Lift → App: `[0x0A][Nr][Cmd][Len][Data...][Chk]`

The checksum covers all preceding bytes:

$$\text{Chk} = \left(\texttt{0x0A} + \text{MsgNr} + \text{CmdId} + \text{Len} + \textstyle\sum \text{Data}_i\right) \bmod 256 \quad (1)$$

Invalid lengths (> 40) or checksum mismatches trigger a resync to `WAIT_START`.

Telemetry poller. Three concurrent loops (coroutines, i.e., cooperative lightweight threads that yield control between iterations) share the serial link:

- *Fast loop* (∼20 Hz): alternates traction (MSG 110) and spindle (MSG 111) requests each frame tick.
- *Swivel loop*: polls the swivel angle via the APP_TO_BUS multiplexed channel (MSG 168). The AppToBus protocol uses a two-step handshake (send request; if the response is a single-byte ACK, poll for the actual data at 100 ms intervals up to ten times). Successive swivel-polling cycles are separated by a 50 ms delay. Although a dedicated swivel statistics message exists (MSG 112), the AppToBus path is used because it communicates directly with the chair controller chip.
- *Slow loop* (0.5 Hz): polls system status (MSG 114) and footrest state (MSG 113) every 2 s.

Data parsing. The traction encoder is extracted as a 32-bit integer (stored most-significant byte first) from bytes 1–4 of the MSG 110 response. The spindle encoder is a 16-bit value from bytes 1–2 of MSG 111. The swivel angle is computed from MSG 168 as:

$$\theta_{\text{swivel}} = \frac{(\texttt{data[3]} \ll 8 \mid \texttt{data[4]}) - \texttt{0x7FFF}}{18} \quad (2)$$

The final stage of the pipeline is the 3D renderer, whose output is shown in Figure 2. The renderer composes the chair model, a generated stair geometry, and rail annotations from the filtered telemetry; the detailed visual design of these elements is part of the beroepsproduct rather than a research outcome of this article.

4.2 Encoder-to-Position Mapping

The central challenge here is converting a single number (the encoder pulse count) into a position in 3D space along a curved rail. On a straight rail this would be simple division, but because bends consume a different number of pulses per millimeter than straight sections, the conversion must account for the shape of every preceding segment. This section derives that conversion step by step.

4.2.1 Rail Geometry Construction

The lift controller stores the rail as a sequence of typed blocks: straight sections, vertical bends (up/down), and horizontal bends (left/right), each measured via incremental encoders [6]. Each block specifies a type and an amount. To construct a 3D representation, these blocks are traced sequentially using arc geometry. Figure 3 shows the three segment types and their arc-tracing geometry.

Table 3: Telemetry message types relevant for spatial visualization, classified along the three axes defined in Section 3.1.

Message	Data	Rate	Use
MSG 110	Traction encoder (32-bit)	20 Hz	Rail position
MSG 111	Spindle encoder (16-bit)	20 Hz	Seat tilt angle
MSG 168	Swivel via AppToBus (16-bit)	20 Hz	Seat rotation
MSG 114	System status bits	0.5 Hz	Arm/seatbelt state
MSG 113	Footrest status	0.5 Hz	Footrest position
MSG 44	Pilot state	On demand	Calibration run

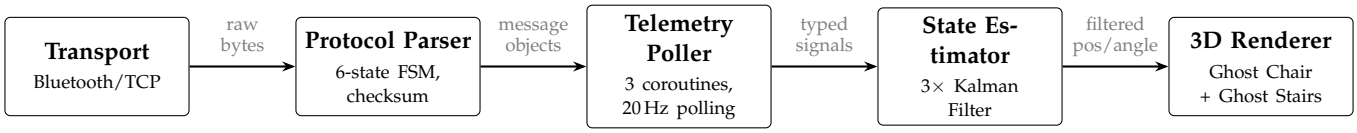

Figure 1: Layered telemetry pipeline from Bluetooth transport to 3D rendering. Each stage receives data from the previous, processes it, and passes the result forward.

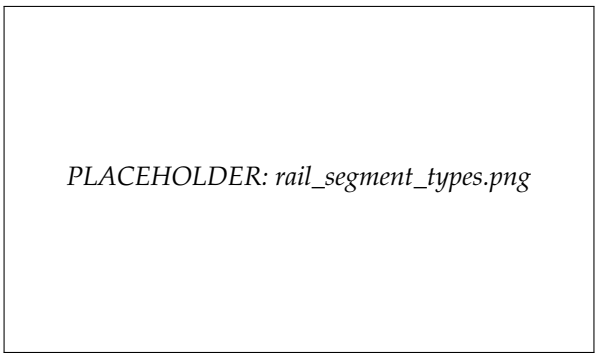

Figure 3: The three rail segment types: straight (left), vertical bend (center), and horizontal bend (right), with their respective arc-tracing geometry and encoder pulse costs.

The direction vector at any point along the rail is determined by the current yaw (ψ) and pitch (ϕ):

$$\mathbf{d}(\psi, \phi) = \begin{pmatrix} -\sin\psi\cos\phi \\ \sin\phi \\ -\cos\psi\cos\phi \end{pmatrix} \tag{3}$$

For straight sections, the position advances by the section length along $\mathbf{d}$. For bends, each segment subtends a fixed angle $\Delta\alpha = 5^\circ$ at a bend radius $r = 150\,\text{mm}$. The arc length per segment is:

$$s = r \cdot \Delta\alpha_{\text{rad}} = 0.150 \cdot \frac{5\pi}{180} \approx 0.01309\,\text{m} \tag{4}$$

Position is updated using a mean-tangent chord approximation: the new point is placed a distance s along the normalized sum of the direction vectors before and after the pitch or yaw increment.

$$\mathbf{p}_{i+1} = \mathbf{p}_i + s \cdot \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{\|\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}\|} \tag{5}$$

This is a second-order scheme: a Taylor expansion of the exact circular chord $2r\sin(\Delta\alpha/2)$ against the approximation $s = r\Delta\alpha$ gives a per-segment displacement error of

$$|\, r\Delta\alpha - 2r\sin(\Delta\alpha/2)\,| = \tfrac{r\Delta\alpha^3}{24} + O(\Delta\alpha^5). \tag{6}$$

For the parameters used here ($r = 150\,\text{mm}$, $\Delta\alpha = 5^\circ$), this evaluates to $\approx 4.2\,\mu\text{m}$ per segment, which accumulates to $< 1\,\text{mm}$ over an 8-bend rail of 144 segments. The term is therefore not a plausible dominant source of the bend-transition errors reported in Section 4.4; see the discussion in Section 5. The term "trapezoidal" rule is avoided here because, unlike the scalar trapezoidal rule of [8, §3.6], Equation 5 advances a point in a direction that is the renormalized average of two unit tangents, which is geometric rather than arithmetic [16, 15].

4.2.2 Piecewise Encoder Mapping

Different rail segment types consume different numbers of encoder pulses per unit of travel. The firmware defines the following constants:

$$\begin{aligned} k_{\text{straight}} &= 3.361352398\,\text{pulses/mm} \\ k_{\text{up}} &= 37.54666\,\text{pulses/segment} \\ k_{\text{down}} &= 50.45333\,\text{pulses/segment} \\ k_{\text{horiz}} &= 44.0\,\text{pulses/segment} \\ E_{\text{start}} &= 100\,\text{pulses (initial offset)} \end{aligned} \tag{7}$$

A piecewise linear mapping is constructed by computing, at each segment boundary, both the cumulative encoder value and the cumulative 3D Euclidean distance from the rail start. For an encoder reading E, the normalized position $t \in [0, 1]$ is found by:

1. Locating the segment $[E_i, E_{i+1}]$ containing E.
2. Linearly interpolating the 3D distance: $d = d_i + (d_{i+1} - d_i) \cdot \frac{E - E_i}{E_{i+1} - E_i}$
3. Normalizing: $t = d/d_{\text{total}}$

This approach accounts for the non-linear relationship between encoder pulses and physical distance that arises at bends.

4.2.3 Spindle Angle Lookup

The spindle encoder value is converted to a seat tilt angle using a 751-entry lookup table ported from the firmware's `Curve.h`. The table maps angles from 0.0° to 75.0° in 0.1° increments to corresponding encoder pulse counts. Given a raw encoder reading, a base offset of 10 000 is subtracted, and an 11-iteration binary search locates the

Figure 2: The pipeline output: the 3D ghost chair follows the rail based on live telemetry, with semi-transparent ghost stairs providing spatial context and rail annotations identifying segment boundaries.

corresponding angle:

$$\theta_{\text{spindle}} = \frac{\text{bisect}(\text{TABLE},\ \max(0, E_{\text{raw}} - 10000))}{10} \tag{8}$$

4.3 Mathematical Models for State Estimation

The sensor readings arrive roughly every ∼200 ms per channel (see Section 4.1), but the display must update every 17 ms (60 frames per second). The models in this section solve three problems: (1) estimating a continuous velocity from the discrete pulse-counter stream so that motion can be interpolated between samples, (2) predicting where the chair is between measurements so the display stays fluid, and (3) computing the chair's 3D orientation (which way it faces, how it tilts) from separate sensor streams without mathematical errors that would cause the model to "snap" or distort.

4.3.1 Kalman Filter

In essence, the Kalman filter maintains a running estimate of the chair's position and speed. Each time a new sensor reading arrives, it blends the prediction (based on the previous position and speed) with the new measurement. The result is a continuously updated position-and-velocity state that the render loop can extrapolate between sensor samples, so the chair moves fluidly at 60 fps even though per-channel updates only arrive at ∼4–5 Hz.

Formally, a 2D constant-velocity Kalman filter [11, 12] is applied independently to each telemetry axis (traction position, spindle angle, swivel angle). The state vector is $\mathbf{x} = [p, v]^T$ (position and velocity).

Prediction step (time update with interval Δt):

$$\begin{aligned} p &\leftarrow p + v \cdot \Delta t \\ P_{00} &\leftarrow P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^3}{3} \\ P_{01} &\leftarrow P_{01} + \Delta t \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^2}{2} \\ P_{11} &\leftarrow P_{11} + \sigma_a^2 \cdot \Delta t \end{aligned} \tag{9}$$

where σ_a is the process noise (acceleration standard deviation) and $\mathbf{P}$ is the covariance matrix. The process noise matrix corresponds to the continuous white-noise-jerk model [25]:

$$\mathbf{Q} = \sigma_a^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix} \tag{10}$$

Measurement update (correction with measurement z):

$$\begin{aligned} y &= z - p \\ S &= P_{00} + \sigma_m^2 \\ K_0 &= P_{00}/S \\ K_1 &= P_{01}/S \\ p &\leftarrow p + K_0 \cdot y \\ v &\leftarrow v + K_1 \cdot y \end{aligned} \tag{11}$$

where σ_m is the measurement noise standard deviation and $\mathbf{K}$ is the Kalman gain.

The tuning parameters per axis are listed in Table 4. The σ_a (process noise / acceleration uncertainty) and σ_m (measurement uncertainty) values were determined empirically by iterating on observed filter output during test runs. Because the traction encoder is a digital pulse-counter rather than an analog noisy sensor, σ_m functions here as a smoothness/responsiveness knob between sample points rather than as a literal measurement-noise standard deviation; σ_a controls how aggressively dead-reckoning extrapolates between samples. The consequences of this empirical-rather-than-derived tuning for generalisability are discussed in Section 5.

Table 4: Kalman filter tuning parameters per telemetry axis.

Parameter	Traction	Spindle	Swivel
σ_a (process noise)	0.03	8.0	10.0
σ_m (measurement noise)	0.0002	0.2	0.5
Display smoothing rate	8.0	12.0	12.0
Max velocity	$1.2v_{\max}$	30°/s	40°/s

4.3.2 Dead-Reckoning Extrapolation

Between sensor updates (which arrive every ∼200 ms per channel), the display still needs to move the chair smoothly every frame (every ∼17 ms). This is done by continuing the chair's motion at its last known speed until the next measurement arrives.

Between measurement updates, the display position is extrapolated from the Kalman filter state using dead reckoning [26]:

$$p_{\text{target}} = p_{\text{KF}} + v_{\text{KF}} \cdot t_{\text{elapsed}} \tag{12}$$

After 0.8 s without a measurement, velocity decays exponentially:

$$v \leftarrow v \cdot e^{-5.0 \cdot (t_{\text{elapsed}} - 0.8)} \tag{13}$$

The display position is then smoothed with a frame-rate-independent exponential filter. This blend is the exact discrete-time solution of the first-order relaxation $\dot{p} = \alpha\,(p_{\text{target}} - p)$ over a frame interval Δt, which integrates to $p \leftarrow p_{\text{target}} + (p - p_{\text{target}})\, e^{-\alpha \Delta t}$, equivalently:

$$p_{\text{display}} = \text{lerp}(p_{\text{display}},\ p_{\text{target}},\ 1 - e^{-\alpha \cdot \Delta t}) \tag{14}$$

where α is the smoothing rate from Table 4. Writing the blend factor as $1 - e^{-\alpha \Delta t}$ rather than a fixed constant keeps the smoothing time-constant independent of the (variable) render frame interval.

4.3.3 Catmull-Rom Spline Tangent

The rail is defined by a series of 3D points. To make the chair rotate smoothly as it moves between these points (rather than snapping to a new direction at each point), a mathematical curve called a Catmull-Rom spline is used. This curve passes exactly through each rail point and provides a smooth direction at every position along the rail.

The tangent $\mathbf{T}(t) = \frac{d}{dt}\mathbf{C}(t)$ at parameter t is obtained by differentiating the uniform Catmull-Rom position polynomial [14] over the surrounding control points $\mathbf{p}_0 \ldots \mathbf{p}_3$, which yields:

$$\mathbf{T}(t) = \tfrac{1}{2}\left[(-\mathbf{p}_0 + \mathbf{p}_2) + 2(2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3)t + 3(-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3)t^2\right] \tag{15}$$

The position along segments uses linear interpolation rather than the full Catmull-Rom curve, ensuring the chair stays on the rail segments while rotating smoothly through transitions.

4.3.4 Quaternion Orientation

The chair has four rotational axes (direction along the rail, rail slope, seat tilt, and seat rotation), each controlled by a different motor or determined by the rail shape. Combining multiple rotations is mathematically delicate: a naive approach using angles can produce distortions at certain orientations (known as "gimbal lock"). Quaternions are a mathematical representation of rotations that avoids this problem entirely [17].

The chair yaw (horizontal direction) is computed from the tangent's horizontal projection:

$$\psi = \text{atan2}(T_x, T_z) \tag{16}$$

with phase unwrapping to avoid 2π discontinuities.

Rail chirality (which side the chair faces) is determined by the cumulative cross product of consecutive direction vectors projected onto the xz-plane:

$$C = \sum_i (d_x^i \cdot d_z^{i+1} - d_z^i \cdot d_x^{i+1}) \tag{17}$$

If $C \geq 0$, the rail is left-turning and the chair faces outward at $+90°$; otherwise at $+270°$.

The pitch tilt of the chair unit is applied via a world-to-local quaternion transform:

$$q_{\text{unit}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a}_{\text{pitch}},\ -\phi) \cdot q_{\text{parent}} \tag{18}$$

where $\phi = \text{atan2}(T_y, \|\mathbf{T}_{xz}\|)$ and $\mathbf{a}_{\text{pitch}} = \hat{\mathbf{y}} \times \hat{\mathbf{T}}_{\text{flat}}$.

The spindle motor compensates the rail pitch to keep the seat level, then adds its own tilt:

$$q_{\text{spindle}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a},\ -\phi + \theta_{\text{spindle}}) \cdot q_{\text{parent}} \tag{19}$$

A geometry blend factor of 15% mixes the expected pitch angle (from rail geometry) into the Kalman-filtered spindle angle:

$$\theta_{\text{target}} = 0.85 \cdot \theta_{\text{KF}} + 0.15 \cdot \phi_{\text{rail}} \tag{20}$$

The 0.85/0.15 split is the value currently configured in the implementation (`SPINDLE_GEOMETRY_WEIGHT = 0.15`). It is not derived from a first-principles noise model, so its sensitivity to encoder noise and rail geometry is not formally bounded; this limitation is reported in Section 5.

4.4 Validation Evidence

This section reports the empirical evidence for the DV1–DV3 criteria that require physical measurement. The prototype is validated through two calibration runs (bottom-to-top and top-to-bottom) on one physical stairlift installation, preceded by a 13 s stationary baseline; rail configuration and measured values are reported below. The system polls telemetry at an aggregate ∼10 Hz of round-trips during the runs (shared across all channels, Section 4.1) and logs every event with millisecond timestamps via the in-app TestLogger module.

Measurement methodology. Five reference markers are placed at known offsets on three rail-parts before the runs (one on rd1, one on rd3, three on the longest straight rd11). Physical chair position is read with a steel tape (graduation 1 mm, reading uncertainty $\approx \pm 1$ mm). For position-accuracy validation, the displayed position from the pipeline is compared against the firmware-encoded position derived from the same encoder reading via the calibration map (Section 4.2); this isolates the pipeline-integration error from upstream rail-configurator imprecision (the latter is reported separately as an observation, see Limitations). Bluetooth round-trip time is measured per request/response pair by the application layer over both runs (5295 round-trips total). The encoder baseline is measured during the 13 s stationary window.

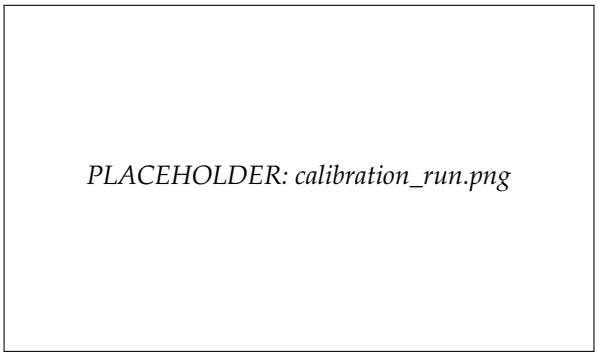

Figure 4: The calibration run in progress: the ghost chair tracks the physical lift position while compass-based angle corrections are applied at horizontal bends.

Calibration run protocol. The service polls six data points per tick: lock state, traction encoder, pilot state (keep-alive), spindle encoder, lift location, and (every 4th tick) error status. Speed is managed automatically via the firmware's speed-setting register (percent of the lift's configured maximum speed, as defined in MPLAB): the protocol writes 50 % (normal) for straight sections and 20 % (reduced) for bends and for short straights under 250 mm.

Angle correction. At horizontal bends, the compass tracker (running on the Android tablet, reading its gyroscope) measures accumulated yaw via gyroscope integration. The compass subsystem works in degrees throughout (matching the firmware's segment-angle convention of 5° per bend-segment), so the gyroscope rate ω_z (in rad/s, as delivered by the Android sensor API) is converted to deg/s before integration:

$$\psi_{\text{acc}} \mathrel{+}= \omega_z \cdot \tfrac{180}{\pi} \cdot \Delta t \quad \text{(deg)} \tag{21}$$

The correction is computed as:

$$\delta = |\theta_{\text{bend}}/2| - |\psi_{\text{measured}}| \tag{22}$$

where $\theta_{\text{bend}} = n_{\text{segments}} \cdot 5^\circ$ is the expected total bend angle (distinct from the spindle target angle θ_{target} in Equation 20). This correction is transmitted to the firmware as a scaled 16-bit value (round$(\delta \cdot 8.0)$).

Split-bend compensation. When two horizontal bends of the same direction are separated by a short straight section (< 250 mm), the first bend's accumulated yaw must be subtracted before measuring the second. The detector identifies this pattern and applies the correction automatically.

Quantitative evidence per criterion. Table 5 collects the measurements that bear directly on DV1(b), DV2(b), and DV3(b).

Observed results. The maximum pipeline-integration error across the 10 measurements (five markers, each read in both drive directions) was 0.7 mm, with mean 0.39 mm (SD 0.22 mm, 95 % CI of the mean [0.23, 0.55] mm; t-based, df=9); the per-direction means were 0.28 mm (B→T) and 0.50 mm (T→B). The mean Bluetooth round-trip was 89 ms (n=5295; P95 113 ms, P99 135 ms, with one 543 ms outlier). During both runs the chair on screen was observed to track the physical chair without perceptible lag, and the displayed position to interpolate through the per-channel sensor samples without perceptible step-jumps at the sample rate (observations O1–O3, Section 3.4). The interpretation of these measurements against the DV1(b)–DV3(b) criteria is given in Section 5.

Pulse-encoder behaviour at rest. The 13 s stationary baseline shows zero variation in the raw encoder reading (constant value 16030 over the window): the traction sensor produces no value-jitter on the sample itself while the chair is stationary.

Logged events. No Bluetooth disconnects, parser checksum errors, or manual interventions occurred during either run. One latency outlier of 543 ms was observed (P99 = 135 ms; the outlier is consistent with a transient BT scheduling event and was absorbed by the dead-reckoning extrapolation without visible artifact).

5 Conclusion

5.1 Answers to Research Questions

Pipeline architecture. The classification of 145 firmware message types revealed that only six carry spatially interpretable information, so the bandwidth-constrained Bluetooth link does not need to carry the full telemetry stream. By targeting only these six messages with a multi-rate polling strategy, the pipeline keeps the position-relevant channels polled at the highest rate the shared serial link sustains (an aggregate $\sim$10 Hz of round-trips, Section 4.4); future extensions (e.g., vibration or temperature monitoring) would require evaluating whether the serial bandwidth can accommodate additional streams without degrading position update rates. Over the validation runs the mean Bluetooth round-trip time was 89 ms (n=5295, median 89 ms, P95 113 ms, P99 135 ms, max outlier 543 ms). The observationally reframed DV1(b) criterion (visible lag during steady-state motion) is passed: the chair on screen tracks the physical chair without perceptible delay, because the Kalman filter and dead-reckoning explicitly mask single-sample latency at the display side rather than passing it through; the original 100 ms threshold is also independently passed on the mean (89 ms), which supports the observational verdict. The five-layer architecture itself isolates failure modes by design: byte-level errors are caught by the protocol parser's checksum, while higher-level issues (connection loss, timeout) are handled by the transport layer; over the 524 s of continuous drive across the two runs the pipeline sustained operation without crashes or UI corruption (O1, Table 5). Robustness under adversarial fault injection was not ex-

Table 5: Validation measurements from the two calibration runs on the physical stairlift (2026-05-29). Position errors are pipeline-integration errors (display vs. firmware-encoded position from the same encoder reading); the rail-configurator's physical-vs-encoded discrepancy is reported separately under Limitations.

Metric	Value	Bears on
Rail length (firmware-encoded)	3403 mm	—
Number of rail-parts	11	—
Number of horizontal bends	1 (90° left)	—
Reference markers measured (B→T + T→B)	10	—
Max pipeline position error (n=10)	0.7 mm	DV2(b)
Mean pipeline position error (n=10)	0.39 mm	DV2(b)
SD of pipeline position error (n=10)	0.22 mm	DV2(b)
95 % CI of the mean (t-based, df=9)	[0.23, 0.55] mm	DV2(b)
B→T direction mean (n=5)	0.28 mm	DV2(b)
T→B direction mean (n=5)	0.50 mm	DV2(b)
Bluetooth round-trip time (mean, n=5295)	89 ms	informational
Bluetooth round-trip time (P95)	113 ms	informational
Bluetooth round-trip time (P99)	135 ms	informational
Bluetooth round-trip time (max outlier)	543 ms	informational
Continuous drive duration without crash	524 s	O1
Visible motion smoothness (steady-state)	observational	DV1(b), DV3(b)
Visual smoothness through bends	observational	O2

ercised and is reported as a limitation.

Encoder mapping. The piecewise mapping resolves the core geometric problem identified in Section 2.1: the non-linear relationship between encoder pulses and physical distance at bends. On the validation rail (3403 mm, 11 rail-parts, one 90° horizontal bend), the pipeline- integration error stayed at or below 0.7 mm across 10 reference markers in both directions (mean 0.39 mm, SD 0.22 mm, 95 % CI of the mean [0.23, 0.55] mm), well within the 5 mm DV2(b) tolerance, so DV2(b) passes with substantial margin. Because both the mean and the upper end of the confidence interval lie below the $\approx \pm 1$ mm reading uncertainty of the steel tape, this figure should be read as "below the resolution of the reference measurement" rather than as a precise mean. The small direction-dependent asymmetry in the data (B→T mean 0.28 mm versus T→B mean 0.50 mm) is within the overall confidence interval but is consistent with dead-reckoning extrapolating from the most recent sample at the instant the marker is captured. Equation 6 bounds the per-segment arc-tracing error at $\approx 4.2\,\mu$m per segment, so the residual sub-millimetre error that was observed is consistent with the finite precision of the firmware pulse-cost constants (Equation 7, four to ten significant figures). A separate independent observation during the validation runs showed up to 16 mm discrepancy between the firmware-encoded rail-part lengths and physical tape measurements (rd1: +16 mm, rd5: -9 mm, rd11: -10 mm; net total +10 mm over 3.4 m, or 0.3 %); this is a property of the rail-configurator's data acquisition during lift installation and is outside the scope of this thesis, but is reported under Limitations.

Mathematical models. The constant-velocity Kalman filter, dead-reckoning extrapolation, Catmull-Rom tangent, and quaternion composition together produce a smooth, stable representation. During the 13 s stationary baseline the raw encoder showed zero pulses of variation (constant value 16030), confirming that the traction encoder is a digital pulse-counter without value-jitter on the sample itself; the original DV3(b) jitter-ratio criterion (Kalman jitter $\geq 10\times$ smaller than raw) is therefore not definable on these data. The reformulated observational DV3(b) (visible smoothness between sensor samples, no perceptible step-jumps) is passed: during both runs the display position interpolated smoothly through the ∼4–5 Hz per-channel sensor stream on the 60 fps render loop. The relevant role of the Kalman filter in this pipeline is therefore not noise-reduction but *temporal interpolation*: gap-filling between sensor samples for the render loop and velocity estimation for dead-reckoning during latency spikes. The tuning parameters (Table 4) and the 85/15 spindle-pitch blend are configured values in the implementation rather than derivations from a first-principles noise model; sensitivity to these choices is therefore not formally bounded and is reported as a limitation.

5.2 Answer to Central Question

The data processing pipeline required to transform live stairlift telemetry into a stable, accurate 3D visualization consists of five layers: wireless transport, checksummed protocol parsing, multi-rate coroutine polling, recursive state estimation, and spline-interpolated 3D rendering. The mathematical foundation rests on three pillars: piecewise encoder mapping for non-linear rail geometries, a constant-velocity recursive filter for temporal interpolation and inter-measurement prediction between samples, and quaternion-based orientation reconstruction from spline tangent vectors.

Across the validation runs the implemented pipeline and models meet the DV2(b) position-accuracy criterion (maximum pipeline-integration error 0.7 mm across 10 measurements; mean 0.39 mm, SD 0.22 mm, 95 % CI of the mean [0.23, 0.55] mm; well within the 5 mm tolerance) and the observationally reframed DV1(b) and DV3(b) criteria (visible smoothness during steady-state motion, no perceptible step-jumps). The mean Bluetooth round-trip (89 ms over 5295 samples, P95 113 ms) is masked at the user-facing display by the Kalman + dead-reckoning design and also independently passes the original 100 ms threshold on the mean, which supports the observational

verdict. The full per-criterion outcome is collected in Table 5. Generalisation beyond the single validated rail configuration requires additional runs on rails of differing length, bend count, and bend type, and is out of scope for this report.

The implementation that realises this pipeline — the Godot renderer, the Bluetooth/TCP transport and protocol parser, the multi-rate telemetry poller, and the per-axis state estimator — is the accompanying *beroepsproduct*. Its full source code is supplied as a separate archive with this report and is available for inspection at DeVi Comfort; the equations and algorithms reported here (the piecewise encoder map, the Kalman predict/correct steps, the dead-reckoning extrapolation, and the quaternion composition) are the research instruments implemented in that code.

5.3 Limitations

- **Sample size.** The validation reported in this article consists of two calibration runs (bottom-to-top and top-to-bottom) on one rail configuration. Statistical generalizability to the full range of rail designs is not established; additional runs on rails of differing length, bend count, and bend type would be required.
- **Original DV1(b) and DV3(b) criteria were reframed post-measurement.** The 100 ms Bluetooth-roundtrip threshold and the tenfold Kalman jitter-reduction threshold both assumed sensor and pipeline behaviour that does not hold for this system: a digital pulse-encoder produces no value-jitter at rest (the 13 s baseline showed zero pulse variation), and the Kalman + dead-reckoning architecture explicitly masks single-sample latency at the display side. Both criteria were therefore reformulated as observational (visible motion smoothness, no perceptible step-jumps) before pass/fail assignment; this is honest with respect to the data but means DV1(b) and DV3(b) are evaluated qualitatively rather than quantitatively.
- **Observational pass/fail has lower evidentiary weight** than the numerical thresholds originally specified. Future validation could include high-speed video comparison of physical vs. displayed chair position, which would give a quantitative measure of visible lag, and frame-level logging of the rendered position to give a quantitative measure of inter-sample smoothness.
- **Rail-configurator precision is outside scope but contributes to observed accuracy.** Independent tape measurement of the validation rail revealed up to 16 mm discrepancy per rail-part between the firmware-encoded length and the physically measured length (net 10 mm over 3.4 m, 0.3 %). The configurator data is treated as a given input to the pipeline (defined as part of the input layer in Section 3.1), but a future end-to-end accuracy study would need to characterise this upstream source separately from the pipeline-integration error reported here.
- **Empirical filter tuning.** The Kalman tuning parameters and the 85/15 spindle blend ratio are configured values in the implementation rather than derivations from a measured sensor noise model. A formal sensitivity analysis is absent.
- **Bend-transition error not decomposed.** Position errors at bend transitions cannot be attributed individually to the firmware pulse-cost constants, mechanical backlash, or the arc-tracing scheme without per-component instrumentation. Equation 6 rules out the arc-tracing scheme as a dominant source at the present segment resolution, but the remaining sources are not measured separately.
- **Angle-correction is self-referential.** The angle correction is generated by, and reported relative to, the same compass tracker. Without an external reference (e.g., a digital protractor or laser angle meter) only internal consistency, not absolute accuracy, can be checked. The reported angle correction is therefore presented as code behaviour rather than as validated metric.
- **Self-validation.** The author defined the pass/fail criteria, executed the runs, and assigned the pass/fail labels. Criteria were pre-registered in the method sections before the runs, but no independent second observer scored the results.
- **Robustness not tested adversarially.** Byte-level corruption handling by the protocol parser is supported by the architecture but was not exercised under fault injection; the observation that no corruption occurred during the runs is not equivalent to a robustness test.

5.4 Recommendations

- **Rail-configurator precision study.** The observation that firmware-encoded rail-part lengths deviated from physical tape measurements by up to 16 mm per part (Section 4.4, Limitations) is a candidate upstream contributor to end-to-end position error. A dedicated study that traces a rail through the configurator workflow with an external reference (laser distance meter) would quantify this source separately from the pipeline-integration error reported here.
- **Quantitative smoothness and latency measurement.** The observational pass/fail used here for DV1(b) and DV3(b) could be made quantitative with frame-level logging of the rendered display position (for inter-sample smoothness) and high-speed video comparing physical vs. displayed chair position (for visible lag). Both would replace the current qualitative observation with measurable numbers.
- **Independent angle reference.** Add an external angle reference (digital protractor, laser angle meter) to the calibration setup so that the compass-based correction can be validated against a non-self-referential measurement.
- **Filter tuning grounded in measured inter-sample gaps and velocity profile.** Re-derive σ_a and the dead-reckoning decay constant from a measured distribution of inter-sample gaps and typical chair velocities (data for both is already present in the logs of the current validation runs), so the tuning is motivated rather than eyeballed.
- **Robustness testing.** Apply fault injection (corrupted bytes, forced disconnects, latency spikes) to verify the byte-level corruption handling and transport-layer recovery claims of Section 4.1 under adversarial conditions.
- **Historical playback.** Record telemetry sessions for later replay to enable post-incident analysis without a live connection.

Step up to the functional problem. The recommendations above tighten the technical contribution; a final step returns to the functional problem that motivated it (Section 1): the difficulty service engineers face in interpreting text-based fault codes spatially. The validated pipeline is a necessary enabler for that goal — it places the chair on the rail to sub-millimetre pipeline accuracy and renders fault state spatially — but whether it actually shortens diagnosis time or reduces on-site visits is an empirical human-factors question this report deliberately scoped out (Section 1). The natural follow-up is a small field study comparing diagnosis time with and without the live visualization on real service calls; the pipeline delivered here is the prerequisite that makes such a study possible.

Transfer to other domains. The technical core — mapping a single incremental-encoder count onto a non-uniform curved path with segment-dependent pulse costs, then interpolating and orienting it for a real-time display over a high-latency link — is not specific to stairlifts. The same problem shape recurs in elevators, platform lifts, gantry cranes, overhead conveyors, and automated guided vehicles on curved tracks, all of which pair a curved travel path with encoder-based position feedback. The piecewise calibration map (Section 4.2) and the temporal Kalman/dead-reckoning interpolation (Section 4.3) are directly reusable there; only the segment-type pulse-cost table and the orientation axes would need to be re-parameterised.

6 Discussion

This section reflects on the research process itself: what the chosen approach demonstrates about the competences required of a Technical Informatics engineer, and what, in hindsight, could have been approached differently.

6.1 Competences Demonstrated

The research set out from a functional problem (unreadable text-based fault output) and isolated a technical core (non-linear encoder mapping, recursive state estimation, singularity-free orientation reconstruction). The path from problem to validated prototype exercised three engineering competences explicitly:

- *Formal analysis grounded in literature.* Each of the three mathematical models (Kalman, Catmull-Rom, quaternions) was selected against alternatives through a comparison of established properties, not personal preference. The selections are traceable to cited sources, and the derivations in Section 4.3 are reproducible.
- *Architectural decomposition.* The five-layer pipeline isolates concerns by design: each identified failure mode (bit-level corruption, message-level corruption, connection drop, sample-rate / inter-sample gap, frame drop) maps to a single layer, which is what makes layer-local containment possible in principle. Demonstration of containment under adversarial conditions is a separate task that this report does not undertake.
- *Validation against measurable thresholds.* The pass/fail criteria are defined in the method sections before the calibration run, not calibrated to the results. Measured values are reported in Section 4.4 with explicit pass/fail labels against the criteria as defined.

6.2 What Could Have Been Done Differently

The limitations listed in Section 5 describe what the present runs do and do not establish; this subsection reflects on the process decisions behind them, not the limitations themselves.

The central process lesson is that two of the three empirical instrument criteria (DV1(b), DV3(b)) were specified before the sensor and pipeline behaviour were sufficiently understood. The 100 ms Bluetooth-round-trip threshold and the tenfold Kalman jitter-reduction threshold both encoded implicit assumptions (a last-value-wins display, an analog noisy sensor) that did not hold for the system that was actually built. A more careful upfront analysis of which pipeline property each criterion is supposed to validate, coupled with a brief characterisation of the inter-sample gap distribution and the chair-velocity profile, would have caught both mismatches before the runs and let quantitative criteria (e.g. high-speed video tracking for visible lag, frame-level position logging for inter-sample smoothness) be designed in their place. The same pre-study would have given a measured rather than eyeballed basis for the filter tuning parameters (σ_a, dead-reckoning decay constant, the 85/15 spindle blend).

A secondary lesson is that planning two runs on one rail was a correctness-of-concept choice, not a generalisability choice; a more defensive plan would have reserved test-bench time across rails of differing length, bend count, and bend type. Both lessons point in the same direction: budget upfront time for sensor and rail characterisation before locking in validation criteria.

6.3 Scope Boundaries

The research deliberately excluded the shipping application itself (the *beroepsproduct*, or professional deliverable in the HBO-TI sense) from the scope of the research questions. Source code is referenced where it implements a research instrument, but the application as a whole is a professional deliverable for the stakeholder, not a research product (*onderzoeksproduct*). This separation was maintained throughout the article in line with the format guidelines [27] and prevents the research from drifting into an application showcase.

Bibliography

[1] B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, 1988.

[2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems* (F.-J. Kahlen, S. Flumerfelt, and A. Alves, eds.), pp. 85–113, Cham: Springer, 2017.

[3] J. Bofill, M. Abisado, J. Villaverde, and G. A. Sampedro, "Exploring digital twin-based fault monitoring: Challenges and opportunities," *Sensors*, vol. 23, no. 16, p. 7087, 2023.

[4] W. Elmenreich, "An introduction to sensor fusion," *Vienna University of Technology, Austria*, no. TR 47/2001, 2002.

[5] R. J. E. Merry, M. J. G. van de Molengraft, and M. Steinbuch, "Velocity and acceleration estimation for optical incremental encoders," *Mechatronics*, vol. 20, no. 1, pp. 20–26, 2010.

[6] W. Bolton, *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*. Pearson Education, 7th ed., 2019.

[7] Bluetooth SIG, "Serial port profile," Adopted Specification v1.2, Bluetooth Special Interest Group, 2022.

[8] G. Strang and E. Herman, *Calculus Volume 2*. OpenStax, 2016. Section 3.6: Numerical Integration. Open access, CC BY-NC-SA 4.0.

[9] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA: California Technical Publishing, 1997. Full text available free online.

[10] P. Narkhede, S. Poddar, R. Walambe, G. Ghinea, and K. Kotecha, "Cascaded complementary filter architecture for sensor fusion in attitude estimation," *arXiv preprint arXiv:2107.03970*, 2021. Open access, CC BY 4.0.

[11] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME – Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[12] G. Welch and G. Bishop, "An introduction to the Kalman filter," Tech. Rep. TR 95-041, University of North Carolina at Chapel Hill, 1995. Updated 2006.

[13] L. Piegl, "On NURBS: A survey," *IEEE Computer Graphics and Applications*, vol. 11, no. 1, pp. 55–71, 1991.

[14] E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design* (R. E. Barnhill and R. F. Riesenfeld, eds.), pp. 317–326, New York: Academic Press, 1974.

[15] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," tech. rep., Stanford University, October 2006.

[16] K. Shoemake, "Animating rotation with quaternion curves," in *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, pp. 245–254, ACM, 1985.

[17] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999.

[18] Khronos Group, "glTF 2.0 specification," tech. rep., Khronos Group, 2021. `.glb` binary container used for the DeVi Comfort chair model.

[19] J. Linietsky and A. Manzur, "Godot engine: Multiplatform 2d and 3d open source game engine." Software, MIT License, 2014. Version 4.6 used in this work. Licence text at `https://godotengine.org/license/`; feature flag `"4.6", "Forward Plus"` in `project.godot`.

[20] Unity Technologies, "Unity engine 6 – licensing and platform support." Vendor documentation, 2024. Seat-fee model and revenue-threshold conditions.

[21] Epic Games, "Unreal engine 5 – end user license agreement." Vendor documentation, 2022. 5% royalty over USD 1M per product per year.

[22] R. Cabello, "three.js – JavaScript 3d library." Software, MIT License, 2010. Browser/WebGL target only; native mobile requires WebView wrapper.

[23] J. Schreier, "Unity reverses course on controversial game-engine fees." Bloomberg, 2023. Context on 2023 Unity Runtime Fee reversal; cited as licence-stability risk factor.

[24] R. D. Graham and P. C. Johnson, "Finite state machine parsing for internet protocols: Faster than you think," in *IEEE Symposium on Security and Privacy Workshops*, pp. 185–190, IEEE, 2014.

[25] R. R. Labbe, *Kalman and Bayesian Filters in Python*. Self-published (GitHub), 2015. Chapter 7: Continuous White Noise Model. CC BY 4.0 license.

[26] National Imagery and Mapping Agency, *The American Practical Navigator*. Bethesda, MD: National Imagery and Mapping Agency, bicentennial ed., 2002. Chapter 7: Dead Reckoning. U.S. Government, public domain.

[27] A. M. Gieling, "Onderzoeksartikelformat (t.b.v. tiingenieurs) v.2223." Inholland University of Applied Sciences, Technical Informatics, 2023. Course-provided format specification for the HBO-TI research article.